

PITCH DECK

ai writes the code.
math approves the merge.



ZK-attested · spec-driven code review for the AI-synthesis era.

AUTHOR

Sumit Vekariya · Solo entry

TRACKS

2 — Specification · 3 — Spec-driven dev

LIVE

veridict.zkred.tech

table of content

01	Hello, here is Veridict
02	Open source is leaking trust
03	The thesis
04	Two primitives
05	How it works
06	What was built
07	Live deployment
08	Results & numbers
09	Trust model
10	Limits & what's next

INTRODUCTION

Most code review today rests on a single named maintainer. AI synthesis makes that worse — review volume now exceeds what any one human can verify by reading. Veridict is a merge gate that decouples **qualified-reviewer-ness** from **identity**. Three formal proofs verify the PR. One ZK signature authorises it. Zero identities revealed.

Live at →

veridict.zkred.tech



hello, here is veridict.

A merge gate for synthesized code. Three formal proofs verify the PR (mypy, pytest, Z3). One Longfellow ZK signature authorizes it. Zero identities revealed.

02 / THE PROBLEM

open source is leaking trust.

Four incidents. One pattern. Different projects, same architectural hole.

INCIDENT 01

Dec 2024

bincode

Maintainer rewrote git history while migrating off GitHub. The questions became doxxing. The project shut down permanently.

INCIDENT 02

Mar 2024

xz-utils

“Jia Tan” earned maintainer status over two years, then shipped a backdoor that nearly compromised sshd on every major Linux distro.

INCIDENT 03

Jan 2024

curl

“My job is increasingly fact-checking the AI,” wrote Daniel Stenberg. Maintainers drown in plausible AI PRs that fail invariants.

INCIDENT 04

Feb 2025

vibe-coding

Karpathy coined it. The practice went mainstream anyway: ship when it looks right. Verification skipped at scale.

Same shape: **identity-coupled trust at the centre · no formal gate before approval · reviewers paying the cost.**

03 / THE THESIS

the thesis.

DECOUPLE

qualified-reviewer-ness from **identity**.

The architectural hole in open-source review is not the absence of clever tooling. It is the lack of separation between being a qualified reviewer and being a known identity.

REPLACE WITH

two separable primitives.

01

Formal gate

mypy + pytest + Z3

02

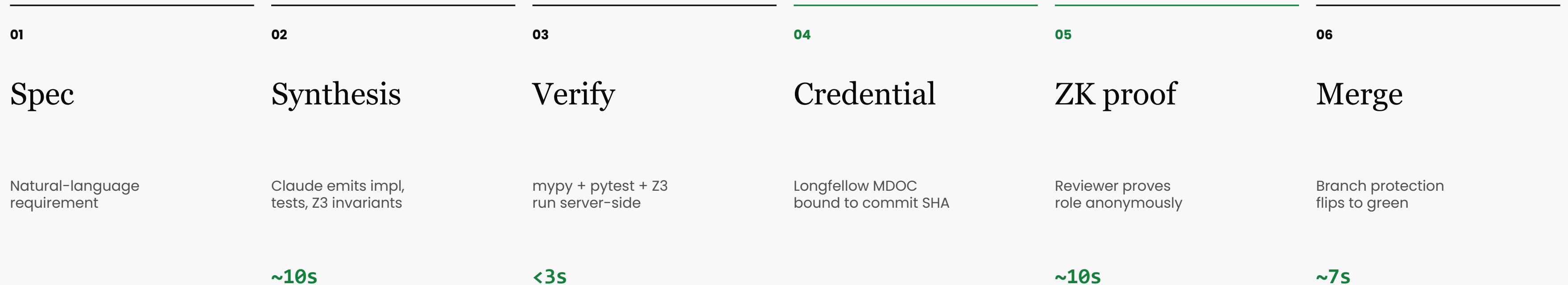
ZK credentials

Longfellow MDOC

04 / HOW IT WORKS

six stages. one signature.

End-to-end on Cloud Run. The reviewer clicks once.



End-to-end **~30 seconds** from PR comment to merge-eligible commit status.

what was built.

Six concrete pieces shipped in the timebox. Every one of them is a real file in the repo.

01 Spec-check gate

mypy + pytest + Z3 results gate ES256-signed MDOC minting.

`issuer/server.py`

03 Longfellow patch

Reviewer namespace org.example.reviewer registered.

`patches/add-reviewer-namespace.patch`

05 GitHub branch gate

Bot pushes zk-review-gate commit status; merge gated at API level.

`.github/workflows/`

02 Structured synthesis

Claude emits impl, pytest suite, and Z3 invariants in one call.

`backend/synth.py`

04 Anti-replay MDOC

Transcript = sha256(owner/repo/pr/sha) bound inside ZK session.

`issuer/mdoc_builder.py`

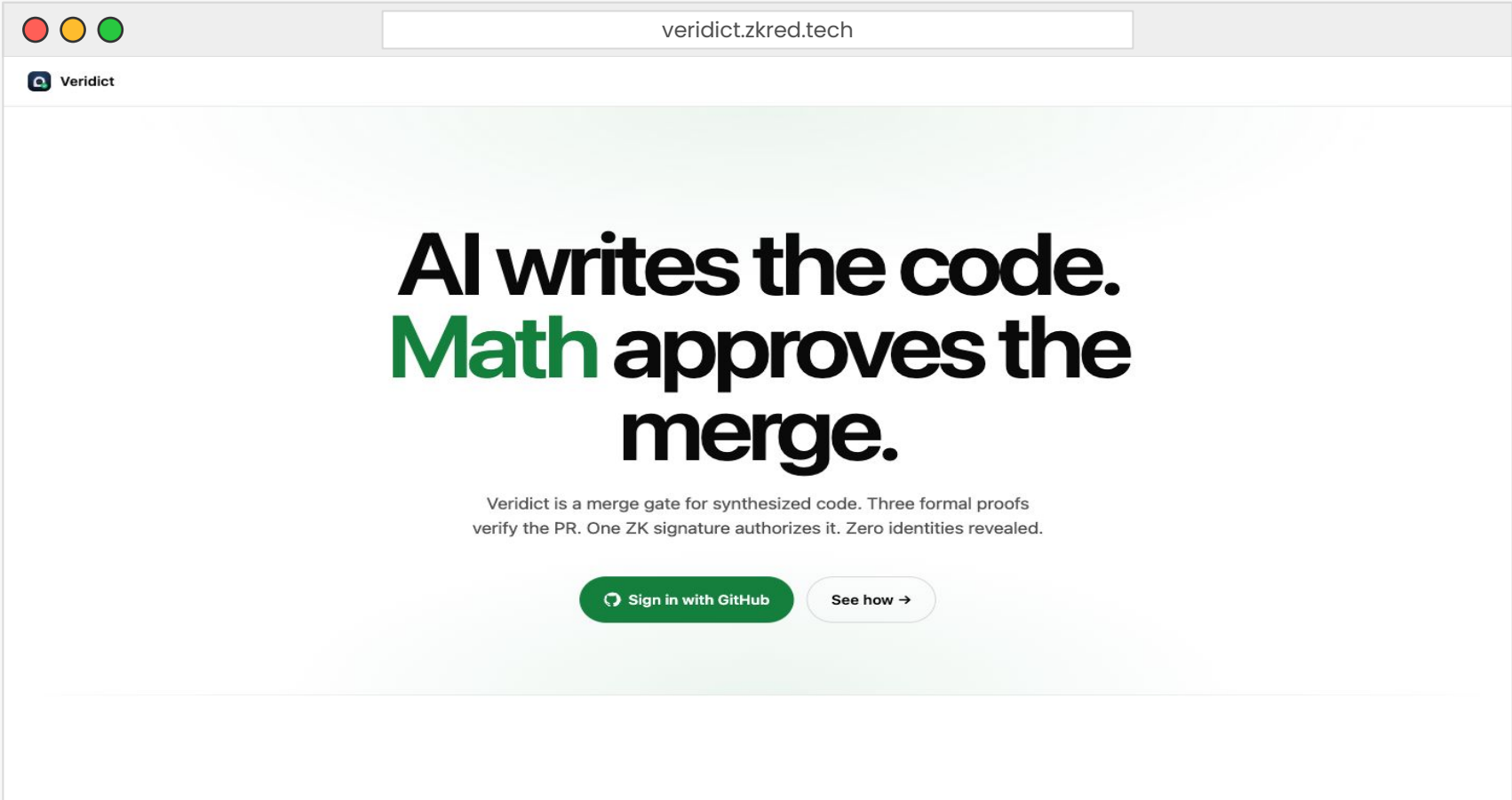
06 Pseudonyms + chips

HMAC-derived per-PR tags; identicons; role/age/repo-count buckets.

`backend/pseudo.py`

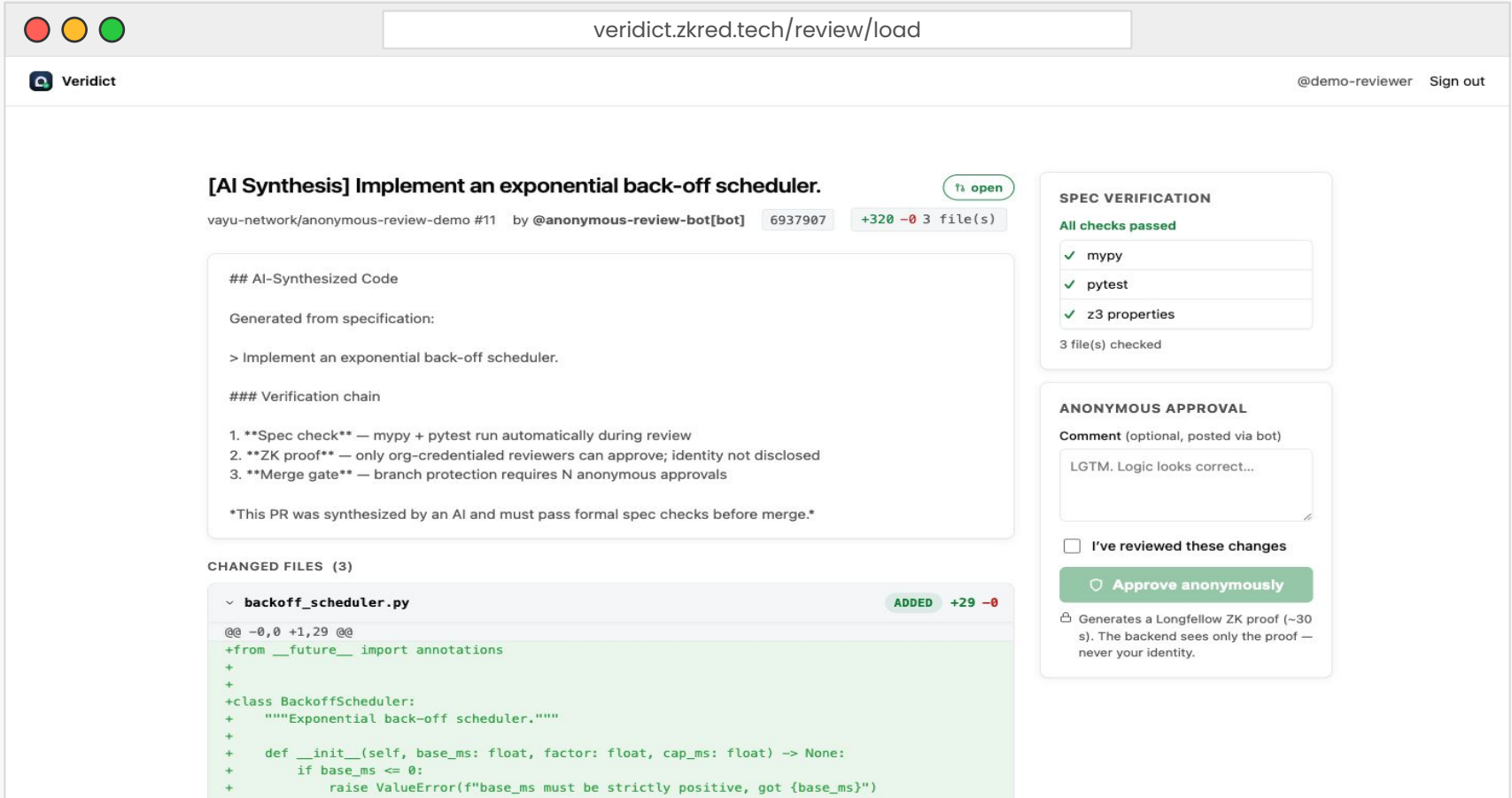
it is real. it is deployed.

Two views from the live deployment — the landing page and the reviewer's PR view.



LANDING

Sign in with GitHub · synth or review.



REVIEWER VIEW

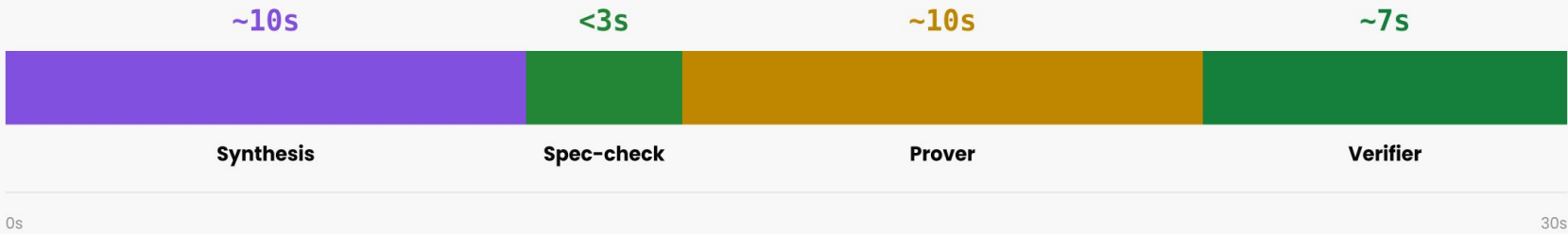
Spec diff, spec-check, approve anonymously.

measured on cloud run.

END-TO-END

~30 seconds

from PR comment to merge-eligible commit status.



PROOF SIZE

360 KB

VERIFIER RSS

365 MB

ANON APPROVALS

2 / 2

PIPELINE TIMINGS

Synthesis	Claude → PR opened	~10s
Spec-check	mypy + pytest + Z3	<3s
Prover	Longfellow ZK	~10s
Verifier	Longfellow ZK	~7s

ARTIFACTS

Proof size	per approval	360 KB
Verifier RSS	peak memory	365 MB
Credential	MDOC, 10-min validity	ES256

DEPLOYMENT

Approvals	anonymous + verified	2 / 2
Branch gate	zk-review-gate context	enforced
Repo	vayu-network/anonymous-review-demo	

who sees what.

Anonymity is structural. Below: who can see which fact, from least private to most public.

	REVIEWER	ISSUER	BACKEND / CI	PR AUTHOR + GH
Reviewer's true identity	• self	✓	—	—
OAuth identity	• own	✓	—	—
Spec-check result	• own PR	✓	✓	✓
ZK proof	• own	—	✓	—
Pseudonym + reputation	• own	✓	✓	✓
Approval count (N / req'd)	—	✓	✓	✓
Other reviewers' identities	—	—	—	—

✓ sees this · • own only (own credential, own PR scope) · — hidden

Note: anonymity is toward the verifier — PR author, CI, audit trail. **Not toward the issuer. Said out loud.**

09 / WHAT THIS ISN'T

limits, said out loud.

A weekend prototype is allowed limits as long as it names them. Here are ours.

ANONYMITY

No nullifier yet

Two browser sessions = two pseudonyms for one reviewer. Fix: derive a blinded ID inside the ZK circuit.

SPEC COVERAGE

Python only, for now

mypy + pytest + Z3 only run on .py files. Non-Python PRs pass through unverified. Dafny, Rust, Lean are obvious next stops.

KEY CUSTODY

Issuer mints the device key

For demo speed. A real deployment generates the device key in the browser so the issuer never sees it.

COMMENTS

Single bot account

Every anonymous approval posts as @anonymous-review-bot. Pseudonyms + identicons inside the comment do the distinguishing.



thank you.

anonymity should be cheaper than identity.

TRY IT

`veridict.zkred.tech`

CODE

`github.com/vayu-network/anonymous-review-demo`